Как SuperTuxKart собирает один кадр

Домашнее задание «Захват кадра» • Dreaaammy • 14.10.2026

Vulkan / RenderDoc 1.24	116 draw calls	83 текстурных ресурсов



Рис. 1. Настоящий кадр №80 игры SuperTuxKart 1.4: трасса Lighthouse, четыре карта, 960 × 540. Изображение выгружено из сохранённого .rdc, не реконструировано по исходникам.

Цель. Проследить путь от вершин и текстур до готовой картинки и проверить, что изменение конкретного GPU-шейдера действительно меняет результат отрисовки.

Среда. Игра и RenderDoc запускались в Debian 12 на удалённом Linux-раннере GitHub Actions. Vulkan обрабатывался программным драйвером Mesa lavapipe (LLVM 15); это реальные команды Vulkan, но не измерение производительности игровой видеокарты. На Mac виртуальная машина не устанавливалась.

Захват и анализ. RenderDoc инъецировался в запущенный процесс игры и захватывал кадр №80. Автогонка профилировщика (--profile-time, --seed=42) делает запуск воспроизводимым; из-за этого HUD показывает 99999 кругов. Данные, доступные в окнах RenderDoc, исследованы через Replay API. Картинки в отчёте - экспорты захвата, а не скриншоты GUI RenderDoc.

Главные находки: один forward-render pass; alpha-test отдельно от прозрачности; позднее рисование skybox; инстансинг; упакованные вершины; BC3 и mipmaps; независимый HUD.

01 / Порядок отрисовки

В захвате один vkCmdBeginRenderPass (EID 6) с очисткой цвета и глубины. Все группы ниже рисуют в одну цветовую картинку ResourceId::129; это логические этапы внутри одного pass, а не пять отдельных Vulkan render passes.



EID 162: непрозрачные объекты и alpha-test; небо ещё не нарисовано.



EID 166: skybox заполнил фон, не перекрыв геометрию.



EID 198: добавлены прозрачные эффекты, HUD отсутствует.



EID 247: поверх 3D-сцены появились мини-карта, время и индикаторы.

EID / draw calls	Что подтверждает pipeline / shader
14-140 / 74	Непрозрачная геометрия: depth LESS, запись глубины, blending выключен.
143-162 / 11	Alpha-test: тот же depth state, но fragment shader отбрасывает alpha < 0.5.
166 / 1	Skybox: 3 вершины, cubemap, depth EQUAL, запись глубины выключена.
171-198 / 15	Прозрачность: depth LESS без записи; ONE / ONE_MINUS_SRC_ALPHA.
205-247 / 15	HUD: depth test выключен; SRC_ALPHA / ONE_MINUS_SRC_ALPHA.
248-250	EndRenderPass → конец command buffer → vkQueuePresentKHR.

Небо рисуется после геометрии: EQUAL оставляет только фоновую глубину. Прозрачные объекты проверяют глубину, но не записывают её; для них используется premultiplied-alpha blending. Отдельных shadow-map, SSAO, bloom, G-buffer или compute-dispatch в этом кадре не обнаружено. Наличие флагов запуска эффектов не доказывает их реализацию в Vulkan-бэкенде 1.4.

02 / Геометрия, инстансинг и буферы

Разобран EID 96: TriangleList, 24 108 индекса, один экземпляр - 8 036 переданных треугольников. Индексы 16-битные. Это число входных примитивов, не число видимых на экране треугольников.

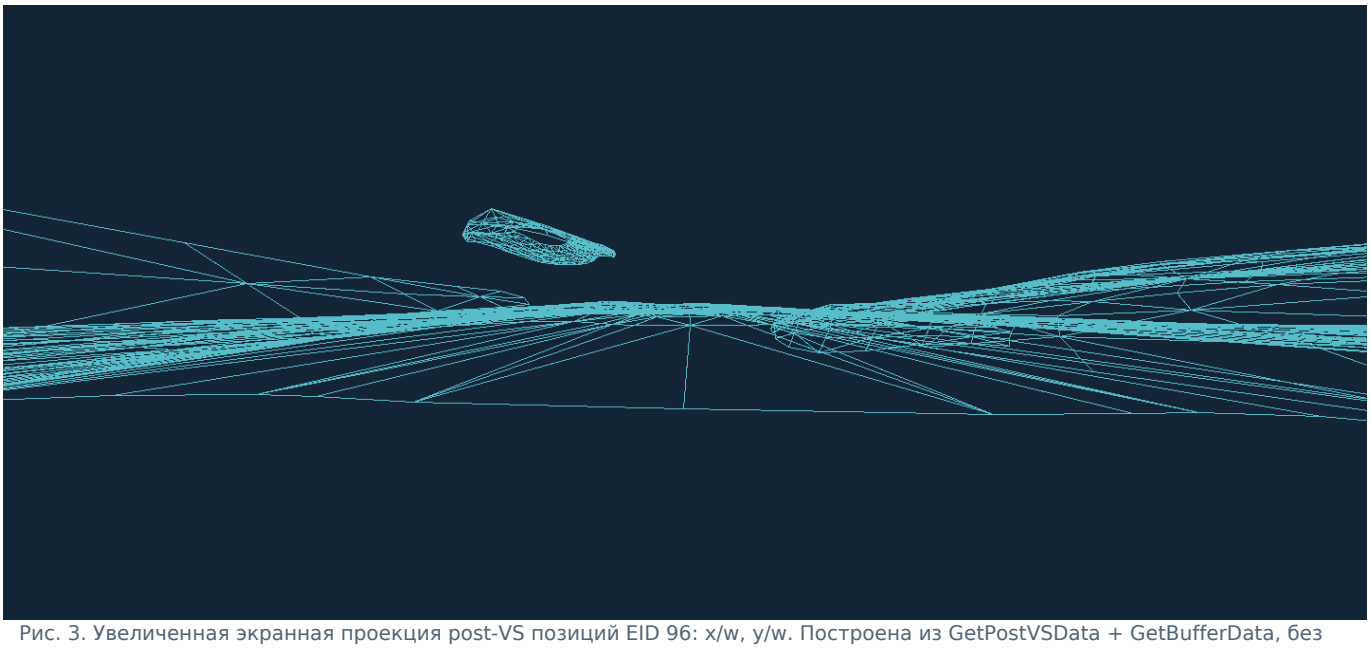


Рис. 3. Увеличенная экранная проекция post-VS позиций EID 96: x/w, y/w. Построена из GetPostVSData + GetBufferData, без depth test, с автоматическим масштабом; невидимые поверхности тоже могут попасть в рисунок. Это визуализация геометрии захвата, не окно Mesh Viewer.

Атрибут	Смещение	Формат GPU
position	0	R32G32B32_FLOAT
normal	12	R10G10B10A2_SNORM
vertex colour	16	R8G8B8A8_UNORM
UV / UV2	20 / 24	R16G16_FLOAT
tangent	28	R10G10B10A2_SNORM

Упаковка. Основная входная вершина занимает 32 байта: UV хранятся в half-float, нормали и тангенты - в упакованном SNORM. В post-VS выгрузке stride уже 64 байта: это выходы шейдера, а не исходный формат vertex buffer. Вершинный и индексный потоки сцены находятся в разных диапазонах одного GPU-буфера ResourceId::3765 (например, EID 49).

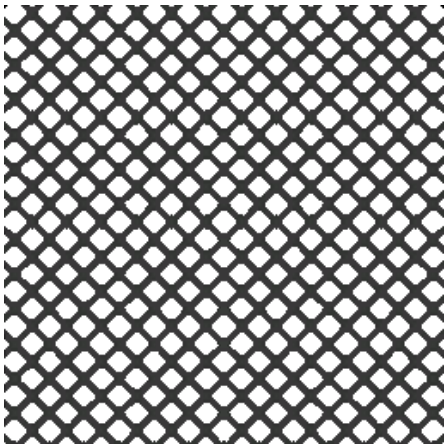
Инстансинг. У 17 draw calls instanceCount > 1: например, EID 52 рисует 14 экземпляров, а EID 173 - 96 экземпляров индексированного quad (6 индексов). В VS 207 gl_InstanceIndex выбирает ObjectData из storage buffer: перенос, поворот, масштаб и параметры материала. Разные объекты можно рисовать одним вызовом без дублирования их сетки.

Константы. UBO u_camera (ResourceId::3850) имеет размер 384 байта: шесть матриц 4 × 4 по 64 байта, включая view, projection и обратные матрицы. Их значения выгружены и доступны в constants.json. Фрагментные и вершинные стадии используют пять разных модулей каждая.

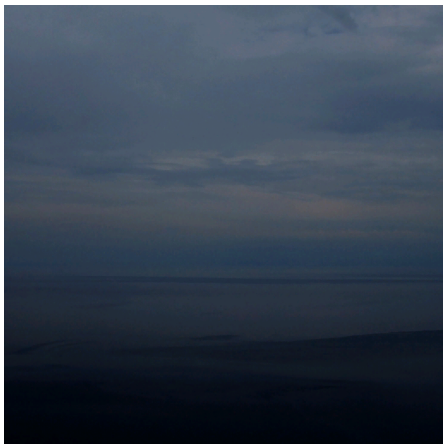
03 / Текстуры и глубина



3880: каменная поверхность
1024², BC3, 11 mip-уровней



3595: alpha-test текстура
256², BC3, 9 mip-уровней



3969: cubemap, грань 0
1024² × 6, BC3, 11 mip-уровней

Рис. 4. Mip 0 и slice 0 реальных изображений. BC3-декодирование выполнено после чтения сырых блоков через GetTextureData; прозрачный фон показан на белом листе.

74 из 83 текстурных ресурсов имеют формат BC3. Блок 4 × 4 занимает 16 байт: для mip 0 изображения 1024² это 1 МиБ вместо 4 МиБ RGBA8 без сжатия. Mip-цепочка 1024 → 1 содержит 11 уровней; она позволяет фильтровать уменьшенные изображения и уменьшает мерцание вдали. 83 - число доступных ресурсов захвата, не число одновременных texture bindings.

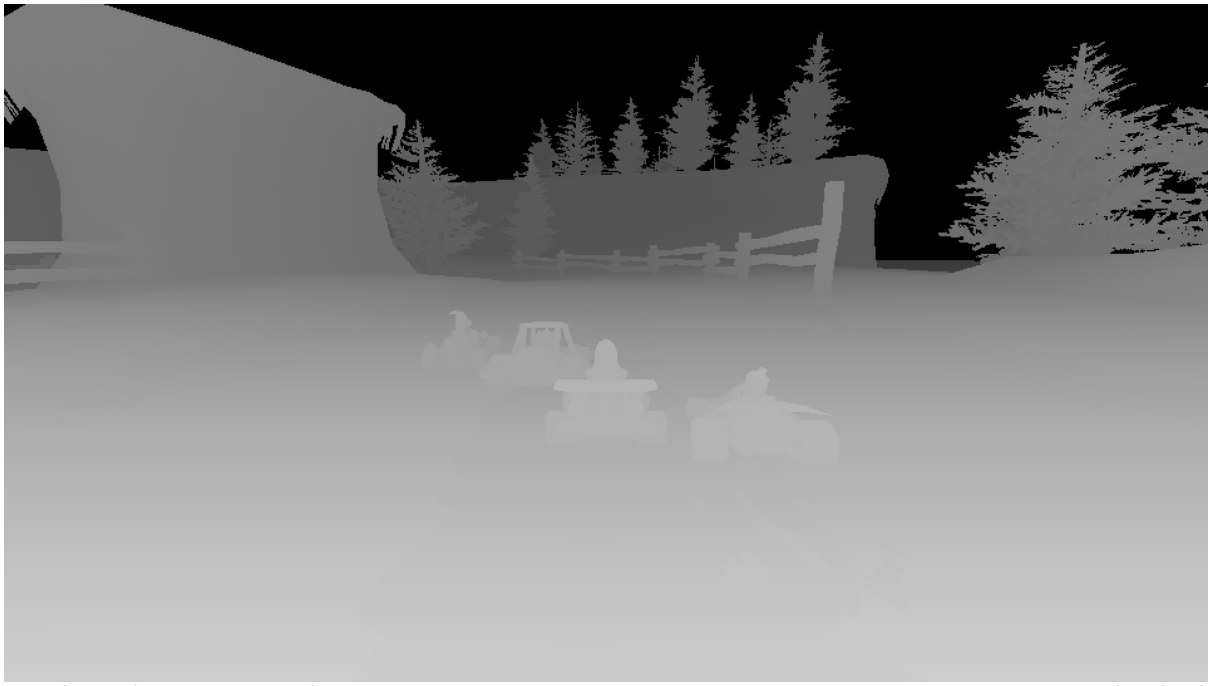


Рис. 5. Depth attachment ResourceId::157, D32, 960 × 540, после EID 198. Для читаемости показана кривая pow(1 - depth, 0.2); это только отображение, не изменение depth buffer.

Alpha-test и blending - не одно и то же. FS 200 делает Kill при textureAlpha × vertexAlpha < 0.5, поэтому отверстия текстуры не записывают ни цвет, ни глубину. FS 209 оставляет непрерывную alpha и смешивается с уже нарисованной сценой. В pipeline у него depth write выключен. У HUD дополнительно выключен depth test.

Цветовая картинка: B8G8R8A8_UNORM, один sample; depth - D32. В данном кадре не найдены HDR color target, MSAA resolve или deferred G-buffer. Отсутствие этих ресурсов относится к этой конфигурации, а не ко всем режимам игры.

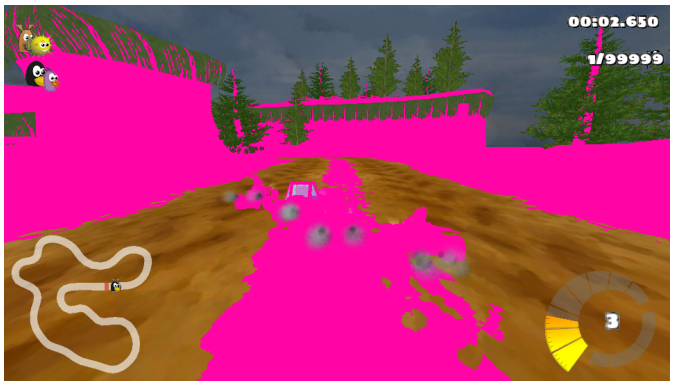
04 / SPIR-V и редактирование шейдера

Модули извлечены из reflection.rawBytes: SPIR-V 1.0, GLSL 450. spirv-dis даёт инструкции байткода, а DisassembleShader RenderDoc - более читаемую запись. Это промежуточный код для Vulkan, не машинный ассемблер конкретной видеокарты.

Модуль / EID	Что удалось прочитать
FS 206 / 49	OpImageSampleImplicitLod + OpFMul: выборка текстуры и умножение на vertex colour; изменение hue через RGB/HSV.
FS 200 / 143	Условное сравнение alpha с 0.5 и OpKill: отбрасывание прозрачных фрагментов.
VS 207 / 49	BuiltIn InstanceIndex, ObjectData и camera matrix: преобразование позиции в clip space.
FS 205 / 166	Выборка cubemap по восстановленному направлению взгляда.



До: исходный replay кадра.



После: замена FS 206; skybox, alpha-test и HUD остаются со своими шейдерами.

В RenderDoc создан SPIR-V replacement через BuildTargetShader, затем ReplaceResource. Новый fragment shader выводит постоянный цвет в location 0:

```
#version 450
layout(location=0) out vec4 colour;
void main() { colour = vec4(1.0, 0.02, 0.65, 1.0); }
```

Проверка результата. Изменились 207 889 из 518 400 пикселей (40,10%). Модуль используется в 74 draw calls, поэтому меняется не только EID 49. После RemoveReplacement исходный RGBA-кадр восстановился побайтово; файл .rdc не изменён.

Окно RenderDoc	Исследованные данные / эквивалент Replay API
Event Browser	Порядок и типы событий: GetRootActions, SetFrameEvent.
Pipeline State	VS/FS, depth, blending, vertex layout, descriptors.
Texture / Resource Inspector	Цвет, depth, mip/slice, форматы и история использования.
Mesh / Buffer Viewer	Индексы, post-VS позиции, vertex stride и camera UBO.
Shader Viewer	Reflection, SPIR-V, disassembly и replacement.

Итог. Прослежен реальный traditional graphics pipeline и выполнено редактирование шейдера с проверкой до/после и отката. Сырые данные, SPIR-V, картинки, скрипты и исходный захват приложены в публичном репозитории; выводы можно перепроверить по EID и ResourceId.

Задание • SuperTuxKart 1.4 • Авторы игры и ассетов • RenderDoc Replay API • Материалы отчёта

SHA-256 исходного .rdc: ec811c7fe011a5527a58b5b83cd9e369292c85eeea36cd7d37a473873aabbbe8